Case Study

Cache Memory Performance Analysis

Student Name	MIMANSHA BHATNAGAR
UID	25BAI10036
Course Name	Computer Architecture and Organization
Faculty Name	DR. RUCHIKA GUPTA
Department	Computer Science and Engineering
University	Chandigarh University

Index

Introduction.....3

Objectives.....4

Problem Statement.....4

Case Study Questions.....5

Q11. Define cache memory.....5

Q12. Distinguish between cache hits and cache misses.....6

Q13. Calculate the cache hit ratio.....7

Q14. Calculate the cache miss ratio.....8

Q15. Why is locality of reference important? 9

Assignment.....10

Relationship Between Hit Ratio and Average Access Time.....12

CPU – Cache – RAM Diagram.....14

Observations.....15

Conclusion.....16

References.....16

Introduction

A modern processor can execute instructions much faster than main memory (RAM) can supply data. If the CPU had to wait for RAM every time it needed an instruction or a data value, it would spend most of its time idle. To reduce this speed gap, computers place a small and very fast memory, called cache memory, between the CPU and the RAM.

The cache keeps copies of the data and instructions that the CPU is likely to need again. When the required data is found in the cache, it is delivered in a very short time. When it is not found, the CPU has to fetch it from the slower RAM, which takes extra time. The performance of the memory system therefore depends on how often the CPU finds what it needs in the cache.

For example, consider the following situation:

Total memory accesses = 1,000

Cache hits = 850

Cache misses = 150

Cache hit time = 2 ns

Additional miss penalty = 50 ns

Using these values, we can measure how effective the cache is and how it affects the average time taken for each memory access.

Objectives

The main objectives of this case study are:

- To define cache memory and explain its role between the CPU and RAM.
- To distinguish between cache hits and cache misses.
- To calculate the cache hit ratio and the cache miss ratio.
- To calculate the Average Memory Access Time (AMAT) using the given formula.
- To measure the effectiveness of the cache.
- To explain the relationship between hit ratio and average access time.
- To explain why locality of reference is important.

Problem Statement

A processor accesses memory 1,000 times. Out of these accesses, 850 are cache hits and 150 are cache misses. The cache hit time is 2 ns and the additional miss penalty is 50 ns. The task is to calculate the cache effectiveness and to explain how the hit ratio is related to the average memory access time.

Case Study Questions

Q1. Define cache memory.

Cache memory is a small, high-speed memory placed between the CPU and the main memory (RAM). It stores copies of the instructions and data that the CPU has used recently or is likely to use soon, so that they can be accessed much faster than from RAM.

Cache memory is needed because the CPU works at a very high speed, while RAM is comparatively slow. The cache acts as a fast buffer between the two.

Important features of cache memory are:

- It is much faster than RAM, because it is built using fast SRAM technology and is placed close to the CPU.
- It is small in size (usually a few KB to a few MB), because fast memory is expensive.
- It stores frequently used data and instructions automatically; the programmer does not manage it directly.
- It is organised in levels: L1 (smallest and fastest), L2 and L3 (larger but slower).
- It reduces the average time the CPU spends waiting for memory.

In this case study, the cache can deliver data in 2 ns, whereas a miss costs an extra 50 ns to fetch data from RAM.

Q2. Distinguish between cache hits and cache misses.

Cache hit: A cache hit occurs when the data requested by the CPU is already present in the cache. The data is supplied directly from the cache in the short hit time (2 ns here).

Cache miss: A cache miss occurs when the requested data is not present in the cache. The CPU must wait while the data is fetched from RAM and copied into the cache. This takes the hit time plus the extra miss penalty (2 ns + 50 ns = 52 ns here).

The differences are summarised below:

Basis	**Cache Hit**	**Cache Miss**
Meaning	Required data is found in the cache	Required data is not found in the cache
Data supplied from	Cache memory	RAM (then copied to cache)
Time taken	2 ns (hit time)	2 + 50 = 52 ns
Number in this case	850 out of 1,000	150 out of 1,000
Effect on performance	Fast; improves performance	Slow; reduces performance
Goal of designer	Increase as much as possible	Reduce as much as possible

Q3. Calculate the cache hit ratio.

The hit ratio is the fraction of memory accesses that are found in the cache.

Formula: Hit Ratio = Number of Cache Hits ÷ Total Number of Memory Accesses

Step 1: Write the given values

- Number of cache hits = 850
- Total memory accesses = 1,000

Step 2: Substitute in the formula

- Hit Ratio = 850 ÷ 1,000

Step 3: Calculate

- Hit Ratio = 0.85
- As a percentage: 0.85 × 100 = 85%

Answer: The cache hit ratio is 0.85 (85%). This means that in 85 out of every 100 accesses, the CPU finds the required data in the cache.

Q4. Calculate the cache miss ratio.

The miss ratio (or miss rate) is the fraction of memory accesses that are not found in the cache.

Formula: Miss Ratio = Number of Cache Misses ÷ Total Number of Memory Accesses

Step 1: Write the given values

- Number of cache misses = 150
- Total memory accesses = 1,000

Step 2: Substitute in the formula

- Miss Ratio = 150 ÷ 1,000

Step 3: Calculate

- Miss Ratio = 0.15
- As a percentage: 0.15 × 100 = 15%

Step 4: Cross-check

- Miss Ratio = 1 − Hit Ratio = 1 − 0.85 = 0.15
- Hit Ratio + Miss Ratio = 0.85 + 0.15 = 1, which is correct.

Answer: The cache miss ratio is 0.15 (15%).

Q5. Why is locality of reference important?

Locality of reference is the tendency of programs to access the same memory locations, or nearby locations, again and again within a short time. It is the main reason why cache memory works well. It has two main types:

- **Temporal locality:** A recently used location is likely to be used again soon, such as a loop counter used in every iteration.
- **Spatial locality:** If one location is used, nearby locations are likely to be used next, such as the elements of an array processed one after another.

Locality of reference is important because:

- The cache holds only a small part of RAM, so it must keep the most useful data. Locality tells us that recently used data is the most useful.
- On a miss, a whole block of neighbouring data is loaded, so spatial locality makes the next accesses likely to be hits.
- Good locality gives a high hit ratio, and a high hit ratio gives a low AMAT.
- Without locality, most accesses would be misses and the cache would give almost no benefit.

In this case study, the 85% hit ratio is possible because the processor's accesses show good temporal and spatial locality.

Assignment

1. Calculate AMAT and Cache Effectiveness

--> Given Information

Total memory accesses = 1,000

Cache hits = 850

Cache misses = 150

Cache hit time = 2 ns

Additional miss penalty = 50 ns

The formula for Average Memory Access Time is:

--> AMAT = Hit Time + (Miss Rate × Miss Penalty)

Step-by-Step Calculation of AMAT

Step 1: Find the Miss Rate

- Miss Rate = 150 ÷ 1,000 = 0.15

Step 2: Find the Miss Penalty Contribution

- Miss Rate × Miss Penalty = 0.15 × 50 ns
- 0.15 × 50 = 7.5 ns

Step 3: Add the Hit Time

- AMAT = 2 ns + 7.5 ns
- AMAT = 9.5 ns

Step 4: Verify using total time

- Time for 850 hits = 850 × 2 = 1,700 ns

- Time for 150 misses = 150 × (2 + 50) = 150 × 52 = 7,800 ns
- Total time = 1,700 + 7,800 = 9,500 ns
- Average time per access = 9,500 ÷ 1,000 = 9.5 ns

Both methods give the same answer, so the calculation is correct.

2. Calculation of Cache Effectiveness

Cache effectiveness shows how much the cache improves memory performance. It is measured here by the hit ratio and by comparing the AMAT with the time needed if there were no cache. For this comparison, we assume that without a cache every access would go to RAM and take 50 ns (the time to fetch data from RAM).

Step 1: Hit ratio

- Hit Ratio = 850 ÷ 1,000 = 0.85 = 85%

Step 2: Speedup due to the cache

- Speedup = Access time without cache ÷ AMAT with cache
- Speedup = 50 ÷ 9.5 = 5.26 (approximately)

Step 3: Percentage reduction in access time

- Reduction = (50 − 9.5) ÷ 50 × 100
- Reduction = 40.5 ÷ 50 × 100 = 81%

Conclusion: The cache makes memory access about 5.26 times faster and reduces the average access time by 81%. If

we compare with the full miss time of 52 ns instead, the speedup is 52 ÷ 9.5 = 5.47, which is almost the same.

Summary of Results

Quantity	**Value**
Hit ratio	0.85 (85%)
Miss ratio	0.15 (15%)
Miss penalty contribution (0.15 × 50)	7.5 ns
Average Memory Access Time (AMAT)	9.5 ns
Speedup compared with no cache	5.26 times
Reduction in access time	81%

Final Output: AMAT = 9.5 ns

Relationship Between Hit Ratio and Average Access Time

The hit ratio and the average memory access time are inversely related. When the hit ratio increases, the AMAT decreases, and when the hit ratio decreases, the AMAT increases.

The reason is simple. A hit costs only 2 ns, but a miss costs 52 ns. If more accesses are hits, fewer accesses pay the extra 50 ns penalty, so the average time becomes smaller. Since Miss Rate = 1 − Hit Ratio, the formula can be written as:

AMAT = 2 + (1 − Hit Ratio) × 50 = 52 − 50 × Hit Ratio

Check with our data: 52 − 50 × 0.85 = 52 − 42.5 = 9.5 ns, which matches the earlier answer. The table below shows how AMAT changes for different hit ratios using the same hit time and miss penalty.

Hit Ratio	**Miss Rate**	**Miss Rate × 50 ns**	**AMAT**
70%	0.30	15.0 ns	17.0 ns
80%	0.20	10.0 ns	12.0 ns
85% (given)	0.15	7.5 ns	9.5 ns
90%	0.10	5.0 ns	7.0 ns
95%	0.05	2.5 ns	4.5 ns
99%	0.01	0.5 ns	2.5 ns
100%	0.00	0.0 ns	2.0 ns

From the table, every 1% increase in hit ratio reduces the AMAT by 0.5 ns (1% of 50 ns). Even when the hit ratio is 100%, the AMAT cannot go below the hit time of 2 ns.

CPU – Cache – RAM Diagram

The following diagram shows how the CPU, cache and RAM are connected and how a memory request moves between them.

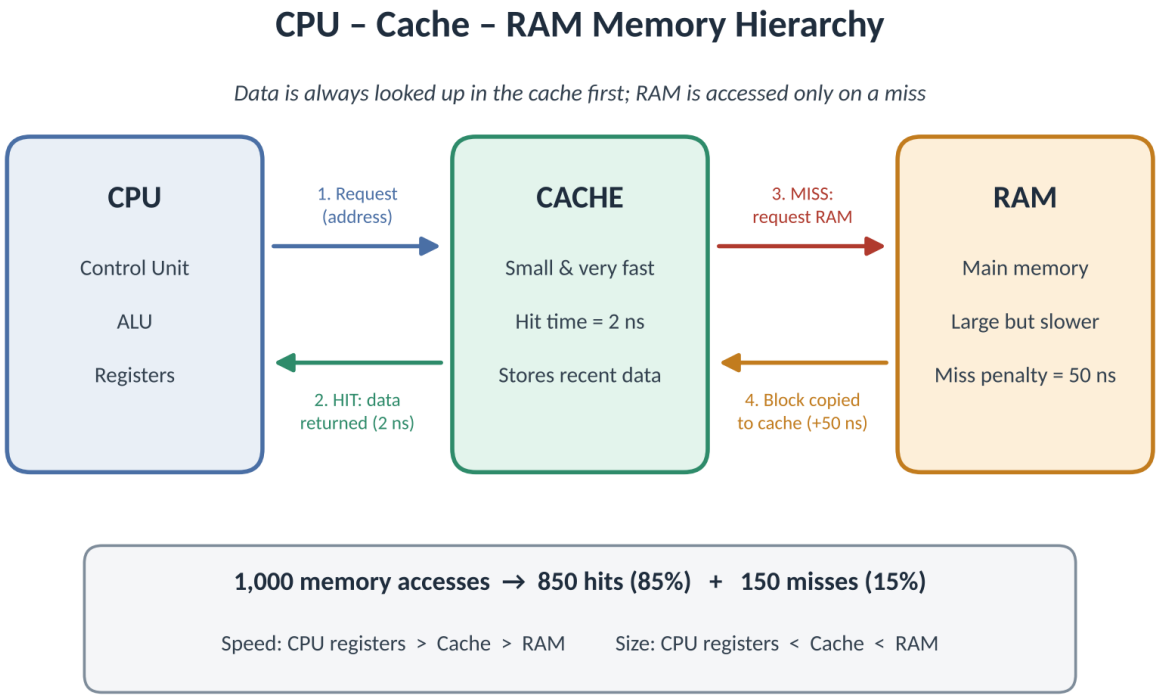


Figure 1: CPU – Cache – RAM memory hierarchy with hit and miss paths

Explanation of Data Flow

1. **Request:** The CPU needs some data and sends the memory address to the cache first.
2. **Cache lookup:** The cache checks whether it holds a copy of that data.
3. **Cache hit:** If the data is present (850 of 1,000 accesses), it is sent to the CPU in 2 ns.
4. **Cache miss:** If the data is absent (150 of 1,000 accesses), the request is passed to RAM.

5. **Block transfer:** RAM supplies the data, which takes an extra 50 ns. The data is copied into the cache and sent to the CPU, so a repeat access will be a hit.

Observations

Based on the calculated results, three observations can be made:

1. **Most accesses are served at cache speed.** With a hit ratio of 85%, 850 of the 1,000 accesses take only 2 ns. As a result, the AMAT is 9.5 ns, which is far below the 52 ns needed for a miss and about 5.26 times faster than using RAM alone.
2. **Misses contribute most of the delay.** Only 15% of the accesses are misses, but they cause 7,500 ns out of the total 9,500 ns, which is about 79% of the total time. This shows that the miss penalty is the main factor that increases the average access time.
3. **A small improvement in hit ratio gives a large gain.** Each 1% increase in hit ratio lowers the AMAT by 0.5 ns. If the hit ratio improves from 85% to 95%, the AMAT falls from 9.5 ns to 4.5 ns, which is a reduction of about 53%. So techniques that improve the hit ratio, such as a larger cache or better program locality, are very valuable.

Conclusion

In this case study, the performance of a cache memory was analysed for a processor that made 1,000 memory accesses. The cache hit ratio was found to be 0.85 (85%) and the miss ratio 0.15 (15%). Using the formula AMAT = Hit Time + (Miss Rate × Miss Penalty), the average memory access time was calculated as 2 + (0.15 × 50) = 9.5 ns, and this was verified by calculating the total time of all accesses.

The cache made memory access about 5.26 times faster than using RAM alone. The study also showed that the hit ratio and the average access time are inversely related, and that locality of reference is the reason a small cache can serve most memory requests. Therefore, a high hit ratio is the key to good memory system performance.

References

1. GeeksforGeeks, “Cache Memory in Computer Organization”.
2. CULMS notes.
3. Gate Smashers, Computer Organisation and Architecture playlist (YouTube).